

Azure Deployment

End-to-end deployment of the Skylar IQ QA Tool to Microsoft Azure

Version	3.6
Updated	May 2026
Audience	Internal team — anyone (no Azure experience required)
Length	~25 pages, ~2-3 hours end-to-end first time

Contents

1.	Overview & what you'll build	3
2.	Architecture at a glance	4
3.	Pre-requisites	5
4.	Phase 1 · Sign in to Azure	6
5.	Phase 2 · Resource Group	7
6.	Phase 3 · PostgreSQL Flexible Server	8
7.	Phase 4 · Azure Container Registry	10
8.	Phase 5 · Build & push the image	11
9.	Phase 6 · Storage & File Shares	13
10.	Phase 7 · Container Apps Environment	15
11.	Phase 8 · Container App	16
12.	Phase 9 · Mount file-share volumes	18
13.	Phase 10 · First sign-in & lockdown	19
14.	Day-to-day operations	20
15.	Cost breakdown	21
16.	Common pitfalls (real ones)	22
17.	Troubleshooting / FAQ	24
18.	Quick reference card	25

1 · Overview & what you'll build

This guide walks you through deploying the **Skylar IQ QA Tool** to Microsoft Azure. By the end you'll have:

- A live HTTPS URL where your QA team can sign in and run automated tests against the Celerant SQL AI Engine and Site Search
- A managed PostgreSQL database for users, runs, and per-query results
- Persistent file storage for uploaded test files, bundles, and generated reports — survives container restarts
- A private container registry that holds your built application image
- A repeatable, version-controlled deployment you can update with one command

What you don't need

- **Docker installed locally** — Azure builds the container image cloud-side from source
- **Kubernetes knowledge** — Azure Container Apps abstracts that away
- **Hand-written ARM/Bicep templates** — every step in this guide is portal click-through or one CLI command

Time & cost

ASPECT	ESTIMATE
Total time, first deployment	2–3 hours (a lot of waiting on Azure provisioning)
Total time, subsequent re-deployments	~5 minutes (just rebuild image + roll new revision)
Approximate monthly cost (idle, ~50 runs/month)	~₹3,700 / ~\$45 USD
Free tier coverage (new Azure account)	\$200 credit ≈ 4 months runway

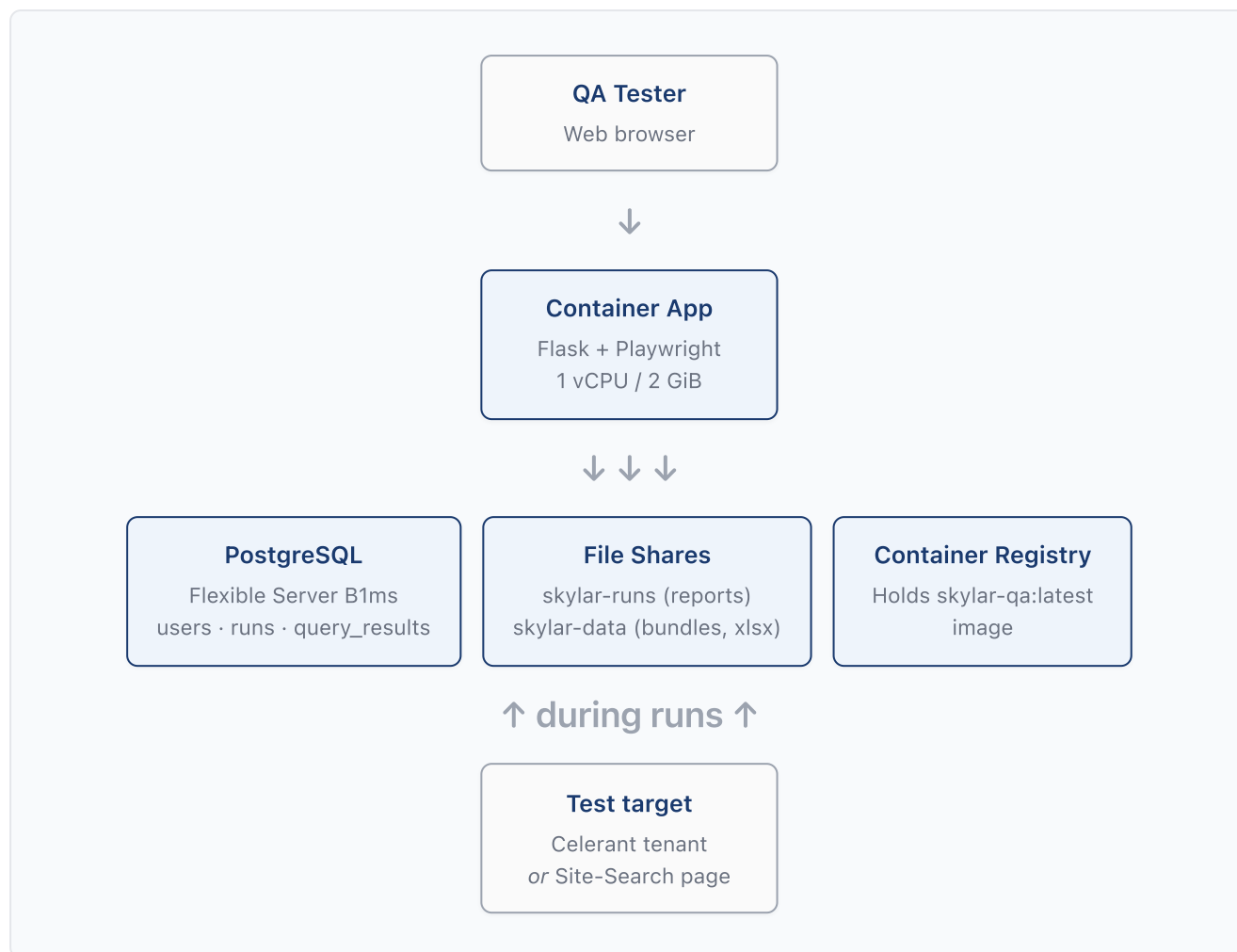
Real-world reference deployment.

Production URL today: <https://ai-platform-auto-tester.thankfulmushroom-67b2c86a.centralus.azurecontainerapps.io/>

Resource group: [ai-platform-auto-tester](#) · Region: **Central US**

2 · Architecture at a glance

Eight Azure resources working together. Read top-to-bottom: a tester's browser hits Azure, which routes to the Container App, which runs Playwright against the test target, and writes results to Postgres + File Shares.



The 8 resources we'll create (in order)

#	RESOURCE	WHY IT EXISTS
1	Resource Group	A folder that holds everything else; deleting it tears down the whole stack
2	PostgreSQL Flexible Server	Persistent database for users, runs, per-query results
3	Azure Container Registry (ACR)	Stores the application Docker image
4	The image itself	Built from source via <code>az acr build</code> ; lives inside ACR
5	Storage Account + two File Shares	Persistent disk for reports (<code>skylar-runs</code>) and uploaded test files / bundles (<code>skylar-data</code>)
6	Container Apps Environment	The shared platform where one or more container apps run; binds the file shares
7	Container App	The actual running app — pulls the image, mounts volumes, exposes HTTPS
8	Volume mounts	The "wires" connecting Container App's <code>/app/runs</code> and <code>/app/data</code> to the file shares

All of these go into a single **Resource Group**. To tear everything down later, delete the resource group — one action, all resources gone.

3 · Pre-requisites

Things you need before starting

ITEM	WHY	HOW TO GET IT
An Azure subscription with credit / payment method	To pay for resources (or use Free Trial credit)	Sign up at azure.microsoft.com/free — \$200 free credit valid 30 days
Azure CLI installed on your Mac	One step (image build) needs it; nice for verification too	<code>brew install azure-cli</code>
The application source code	To build the image from code	<code>git clone https://github.com/farhan713/ai-platform-auto-tester.git</code>
~30 GB free disk space	Source + Docker daemon + Azure CLI cache	—
Internet connection	You'll be uploading source to Azure	Wired/strong WiFi recommended (we hit upload timeouts on slow connections)

⚠ Common false start: account without a subscription.

Just creating a Microsoft account does **not** activate an Azure subscription. You must go through the Free Trial sign-up flow which requires phone OTP **and a credit card** (the card is for identity verification only — won't be charged unless you upgrade). If you skip this step, every Azure CLI command will fail with `No subscriptions found for your-email`.

PHASE 1 OF 10 · ~5 MINUTES

Sign in to Azure

creates: nothing

cost: \$0

tools: CLI + browser

1 Install Azure CLI (one-time, on your Mac)

```
brew install azure-cli
```

Verify it landed:

```
az --version
```

You should see `azure-cli 2.x.x`.

2 Start the device-code login flow

```
az login --use-device-code
```

The command prints something like:

```
To sign in, use a web browser to open the page  
https://login.microsoft.com/device and enter the code XXXXXXXXX
```

3 Complete sign-in in your browser

1. Open `https://login.microsoft.com/device`
2. Enter the 9-character code shown in the terminal
3. Sign in with the **account that has the active subscription**
4. Click **Continue / Allow**

Within 1–2 seconds the terminal will show your subscription details.

4

Verify you're on the right subscription

```
az account show --query '{Name:name, ID:id, State:state, User:user.name}' -o table
```

Expected output:

Name	ID	State	User
Azure subscription 1	8fb70e2e-0f55-4708-bd04-7ea4f38f6147	Enabled	you@work.com

If **State** is anything other than **Enabled**, your subscription isn't active — go back and complete Free Trial activation first.

5

Register the resource providers (one-time per subscription)

```
az provider register --namespace Microsoft.App
az provider register --namespace Microsoft.OperationalInsights
az provider register --namespace Microsoft.DBforPostgreSQL
az provider register --namespace Microsoft.ContainerRegistry
az provider register --namespace Microsoft.Storage
```

Wait ~2 minutes, then verify all five show **Registered** :

```
az provider list --query "[?namespace=='Microsoft.App' ||
namespace=='Microsoft.OperationalInsights' ||
namespace=='Microsoft.DBforPostgreSQL' || namespace=='Microsoft.ContainerRegistry'
|| namespace=='Microsoft.Storage'].{Provider:namespace, State:registrationState}" -o
table
```

Resource Group

creates: 1 RG cost: \$0 (RGs are free) tools: portal

The Resource Group is the container that holds every other resource. We'll name it `ai-platform-auto-tester` in our reference deployment.

1 Open the portal

Go to <https://portal.azure.com> and sign in with the same account you authenticated the CLI with.

2 Create the resource group

1. Top search bar → type "**Resource groups**" → click the result
2. Click **+ Create** at the top of the page
3. Fill in:
 - **Subscription:** `Azure subscription 1` (or whichever)
 - **Resource group:** `ai-platform-auto-tester`
 - **Region:** **Central US** (or your team's preferred region — write it down, every other resource must use the same one)
4. Click **Review + create** → **Create**

Provisions in ~10 seconds. You can leave the page; we'll come back to this RG over and over.

Naming convention.

Throughout this guide we use `ai-platform-auto-tester` as the resource group name. If you choose a different name, substitute it in every `-g <rg-name>` CLI command later. We strongly recommend keeping the name as-is for your first deployment so the reference snippets work verbatim.

PostgreSQL Flexible Server

creates: Postgres server + database

cost: ~₹1,800/mo (Burstable B1ms)

tools: portal

The persistent database. Holds user accounts, run history, per-query results, presets, test-file metadata, hosted-bundle metadata.

1 Start the create flow

1. Top search → "Azure Database for PostgreSQL flexible servers" → click the result
2. Click **+ Create** → choose **Flexible server**

2 Fill the Basics tab

FIELD	VALUE
Subscription	Azure subscription 1
Resource group	ai-platform-auto-tester
Server name	ai-platform-auto-tester-db-XXXX (must be <i>globally</i> unique — append digits if taken)
Region	Central US (same as RG)
PostgreSQL version	16
Workload type	Development (picks <i>Burstable B1ms</i> — cheapest tier; production switches you to <i>Memory-Optimized</i> which is ~5× the cost)
Authentication method	PostgreSQL authentication only
Admin username	CelerantAITesting (or any name; remember it)
Admin password	8+ chars, mix of upper/lower/digit, avoid < > ' " \$ — save this in a password manager immediately

3

Configure networking

1. Click the **Networking** tab at the top
2. **Connectivity method:** select **Public access (allowed IP addresses)**
3. Tick: **Allow public access from any Azure service within Azure to this server**

This lets the Container App (also on Azure) reach the database without VNet plumbing. For a production-hardened setup you'd switch to Private endpoint, but Public + Azure-services-only is plenty secure for QA.

4

Click Review + create → Create

Provisioning takes **3–5 minutes**. Don't refresh the page; just wait. Use this time to start Phase 4 (ACR) in another tab.

5

Add the application database

Once the server is Ready, open it.

1. Left sidebar → expand **Settings** → click **Databases** (if you don't see it, the Settings group may need to be clicked first to expand)
2. Click **+ Add**
3. Enter database name: `auto_tester` (or `skylar` ; remember which)
4. Click **Save**

6

Note these values for Phase 8

Open the Postgres server's **Overview** tab. You'll need:

Server name	<code>ai-platform-auto-tester-db-XXXX.postgres.database.azure.com</code>
Admin username	<code>CelerantAITesting</code>
Database	<code>auto_tester</code>
Password	(the strong password you saved in Step 2)

Pitfall: where is "Databases" hidden?

Azure recently moved the Databases option inside **Settings** in the left sidebar. If you don't see it as a top-level entry, click **Settings** first to expand the group — **Databases** is the 6th item. Alternative: use the portal's Cloud Shell with `az postgres flexible-server db create -g ai-platform-auto-tester --server-name <your-server> --database-name auto_tester`.

Azure Container Registry (ACR)

creates: 1 ACR

cost: ~₹400/mo (Basic SKU)

tools: portal

ACR is your private Docker image registry. Phase 5 will push the application image into it; Phase 8 will pull from it.

1 Start the create flow

1. Top search → **"Container registries"** → **+ Create**

2 Fill in Basics

FIELD	VALUE
Resource group	<code>ai-platform-auto-tester</code>
Registry name	<code>aiplatformautotester</code> (lowercase only, globally unique, no hyphens)
Location	Central US (same as everything)
Pricing plan	Basic (Standard/Premium add geo-replication, security scanning — overkill for QA)

3 Review + create → Create

Provisions in ~1 minute.

4

Enable Admin user

Once created, open the registry.

1. Left sidebar → **Settings** → **Access keys**
2. Toggle **Admin user: Enabled** → click **Save**

This lets the Container App authenticate to the registry using a username/password (instead of managed identity, which is more setup).

5

Note these values for Phase 8

Login server	aiplatformautotester.azurecr.io
Username	aiplatformautotester
Password	password (from Access keys page)

PHASE 5 OF 10 · ~3 MINUTES (FAST CONNECTION)

Build & push the application image

creates: 1 image in ACR

cost: \$0 (build minutes free on Basic)

tools: CLI

This is the only step that **must** be done from the command line. The `az acr build` command uploads your source code, builds the Docker image cloud-side, and stores it in your registry — without needing Docker installed locally.

1 Open Terminal on your Mac

Press **⌘ + Space**, type **Terminal**, hit Enter.

2 Clone the source code

```
cd ~
git clone https://github.com/farhan713/ai-platform-auto-tester.git
cd ai-platform-auto-tester
```

Confirm you're in the right place:

```
ls
# Should show: app/ Dockerfile deploy/ requirements.txt README.md ...
```

3 Build & push the image (one command)

```
az acr build --registry aiplatformautotester --image skylar-qa:latest .
```

The trailing `.` tells `az` to use the current directory as build context.

You'll see Docker layer output stream by. The whole process takes **2–4 minutes**. Last lines you want to see:

```
2026/05/07 09:30:36 Successfully pushed image: aiplatformautotester.azurecr.io/skylar-qa:latest
Run ID: cj1 was successful after 2m49s
```

4

Verify the image landed

```
az acr repository show-tags --name aiplatformautotester --repository skylar-qa -o table
```

Output:

```
Result
-----
latest
```

⚠ Pitfall: upload timeout.

On a slow connection, `az acr build` can hit a write timeout while uploading the source archive. The error looks like:

```
ERROR: ('Connection aborted.', TimeoutError('The write operation timed out'))
```

Cause: a stale `runs/` folder (per-job test artefacts from running locally) gets included in the upload — easily 300+ MB. Fix:

```
mv runs /tmp/runs.bak    # move local test outputs aside
mkdir runs               # keep an empty placeholder so the app still works locally
az acr build --registry aiplatformautotester --image skylar-qa:latest .
mv /tmp/runs.bak runs    # restore after the build succeeds
```

Then re-run the build. With `runs/` moved aside, the upload is <1 MB and finishes in seconds.

Future updates.

To redeploy after pulling new code:

```
git pull
az acr build --registry aiplatformautotester --image skylar-qa:latest .
az containerapp update -g ai-platform-auto-tester -n ai-platform-auto-tester --image
aiplatformautotester.azurecr.io/skyler-qa:latest --revision-suffix "v$(date +%s |
tail -c 5)"
```

The Container App rolls out the new revision with zero downtime.

Storage Account & File Shares

creates: 1 storage acct + 2 file shares

cost: ~₹150/mo

tools: portal

The application needs persistent disk for two folders: `/app/runs` (generated reports + screenshots) and `/app/data` (uploaded test files + ssLibrary bundles). Without these, every container revision rollout would wipe everything users uploaded. We use Azure File Shares (SMB-mounted into the container).

1 Create the storage account

1. Top search → "Storage accounts" → + Create

2. Fill in **Basics**:

RESOURCE GROUP	ai-platform-auto-tester
STORAGE ACCOUNT NAME	aiplatformautotester (lowercase, globally unique — pick something else if taken)
REGION	Central US
PERFORMANCE	Standard
REDUNDANCY	LRS (Locally-redundant storage — cheapest)

3. Click **Review + create** → **Create** (~30 sec)

2 Create the first file share

1. Once created, click **Go to resource**

2. Left sidebar → **Data storage** → **File shares**

3. Click **+ File share**

4. Fill:

- **Name:** skylar-runs
- **Access tier:** Hot
- **Quota:** 5 GiB

5. **Create**

3 Create the second file share

Repeat Step 2 with a different name:

- **Name:** `skylar-data`
- **Access tier:** Hot
- **Quota:** 5 GiB

You should now see both shares listed:

skylar-runs	For <code>/app/runs</code> (reports, screenshots)
skylar-data	For <code>/app/data</code> (uploaded xlsx + bundles)

4 Get the storage account access key

You'll need this to bind the file shares to the Container Apps Environment in Phase 7.

1. Same storage account → left sidebar → **Security + networking** → **Access keys**
2. Click **Show** next to **key1**
3. Click the copy icon next to the long base64 **Key** string
4. Paste it into a private notes app — you'll need it once in Phase 7

⚠ Don't paste the access key anywhere public.

This key gives full read/write access to your storage account. Treat it like a password. Once you've used it in Phase 7, you can stop carrying it around — Azure remembers the binding from there.

Why two file shares, not one?

They have different lifecycle and backup needs. `/app/runs` grows with every test run (screenshots can be a few MB each); easy to grow or rotate. `/app/data` is small, important user-uploaded content that you might want to back up before destructive changes. Keeping them separate lets you tune quota and retention per concern.

Container Apps Environment

creates: 1 env + 2 storage bindings

cost: \$0 (env itself is free)

tools: portal

The "environment" is the platform layer that hosts one or more Container Apps. It manages networking, log aggregation, and shared storage bindings.

1 Create the environment

Two ways to land on the create form:

- **Direct URL:** <https://portal.azure.com/#create/Microsoft.ContainerAppEnvironment>
- **Or:** Home → + **Create a resource** → search "**Container Apps Environment**"

Fill in **Basics**:

RESOURCE GROUP	ai-platform-auto-tester
ENVIRONMENT NAME	skylar-qa-env (or accept the auto-generated name)
REGION	Central US
WORKLOAD PROFILES	Leave default — Consumption only
NETWORKING	Leave defaults — auto-create VNet

Click **Review + create** → **Create**. Provisioning takes **~2 minutes**.

2 Bind the FIRST file share to the environment

1. Once created, click **Go to resource**
2. Left sidebar → **Settings** → **Azure Files**
3. Click **+ Add**
4. Fill in:
 - **Name:** `skylar-runs-mount`
 - **Storage account name:** `aipatformautotester` (pick from dropdown)
 - **Storage account key:** paste the access key from Phase 6 Step 4
 - **File share:** `skylar-runs` (pick from dropdown — it lists shares from the storage account)
 - **Access mode:** **Read/Write**
5. Click **Add**

3 Bind the SECOND file share

Repeat Step 2 with the second share:

- **Name:** `skylar-data-mount`
- **File share:** `skylar-data`
- Same key, same Read/Write access

You should now see both bindings listed.

⚠ Pitfall: order matters.

The Container App's **Volumes** dropdown in Phase 9 only shows file shares that are **already bound at the env level**. If you skip Phase 7 Step 2/3 and try to add a volume to the app first, the dropdown will be empty with the message `No file shares found on this Environment`. Bind first, mount second.

Container App

creates: the running app

cost: ~₹2,500/mo (1 vCPU/2 Gi)

tools: portal

This is where the app actually starts running. The Container App pulls the image from ACR, sets environment variables (including the database connection string), and exposes an HTTPS URL.

1 Start the create flow

1. Top search → **"Container Apps"** (no "Environments" suffix)
2. Click **+ Create** → choose **Container App**

2 Basics tab

RESOURCE GROUP	ai-platform-auto-tester
CONTAINER APP NAME	ai-platform-auto-tester (or any name)
REGION	Central US
CONTAINER APPS ENVIRONMENT	Pick the env you just created (skylar-qa-env or auto-generated name)

3 Container tab

1. Use quickstart image: **NO** (untick)
2. Image source: **Azure Container Registry**
3. Registry: **aiplatformautotester** (pick from dropdown)
4. Image: **skylar-qa**
5. Image tag: **latest**
6. CPU and memory: **1.0 CPU cores · 2.0 Gi memory** (below 2 GiB, Chromium will OOM)

4

Environment variables

Click **+ Add** three times:

NAME	SOURCE	VALUE
<code>DATABASE_URL</code>	Manual	<code>postgresql://CelerantAITesting:<PG-PASSWORD>@ai-platform-auto-tester-db-XXXX.postgres.database.azure.com:5432/auto_tester?sslmode=require</code>
<code>SQA_SECRET_KEY</code>	Manual	A random 64-char hex string. Generate with: <code>openssl rand -hex 32</code>
<code>SQA_ALLOW_SIGNUP</code>	Manual	<code>true</code> (set to <code>false</code> later, after your QA team has accounts)

⚠ URL-encode special characters in the password.

If your Postgres password contains `@ # % / : ? & +` or a space, you must URL-encode them in the `DATABASE_URL` :

<code>@ → %40</code>	<code># → %23</code>	<code>% → %25</code>	<code>/ → %2F</code>
<code>:</code> → %3A	<code>? → %3F</code>	<code>& → %26</code>	<code>+</code> → %2B

Plain alphanumeric + `! _ - * ( )` are safe to use as-is. If you used only those in your password, no encoding needed.

5

Ingress tab

1. **Ingress:** ☒ Enabled
2. **Ingress traffic:** Accepting traffic from anywhere
3. **Ingress type:** HTTP
4. **Target port:** `5050`

6

Review + create → Create

Provisioning takes ~**2 minutes**. The first revision boots and starts serving traffic.

Mount file-share volumes

creates: 2 volumes + 2 mounts

cost: \$0

tools: portal

The Container App is running, but its `/app/runs` and `/app/data` are on ephemeral container disk — every revision rollout would wipe them. We now wire those paths to the persistent file shares we bound to the env in Phase 7.

1 Add the first volume (runs)

1. Open the Container App
2. Left sidebar → **Application** → **Volumes**
3. Click **+ Add**
4. Fill:
 - **Name:** `runs-vol`
 - **Volume type:** Azure file volume
 - **File share name:** `skylar-runs-mount` (*picks from dropdown — populated from Phase 7*)
 - **Mount options:** leave empty
5. Click **Add**

2 Add the second volume (data)

Click **+ Add** again:

- **Name:** `data-vol`
- **File share name:** `skylar-data-mount`

1. Same Container App → left sidebar → **Application** → **Containers**
 2. Tick the checkbox next to your container row → click **Edit and deploy** at the top
 3. In the right pane, scroll to the **Volume mounts** section → click **+ Add new volume mount**
 4. First mount:
 - **Volume name:** `runs-vol`
 - **Mount path:** `/app/runs`
 5. Click **+ Add new volume mount** again. Second mount:
 - **Volume name:** `data-vol`
 - **Mount path:** `/app/data`
 6. Click **Save** in the right pane
 7. Click **Create** at the bottom of the page — this creates a new revision with both mounts wired up
- Wait ~1 minute for the new revision to roll out. The page will show the new revision name (e.g. `ai-platform-auto-tester--XXXX`) and turn **Healthy**.

What you just did.

All four artefact stores now persist across revision rollouts and container restarts:

users / runs / per-query results	→ PostgreSQL
screenshots / REPORT.html / per-query JSONs	→ <code>skylar-runs</code> file share
uploaded test xlsx / hosted bundles	→ <code>skylar-data</code> file share

First sign-in & lockdown

creates: first user account cost: \$0 tools: browser

1 Get the application URL

1. Open the Container App → **Overview** tab
2. Look for **Application URL** in the top-right
3. It looks like: `https://ai-platform-auto-tester.<random>.centralus.azurecontainerapps.io`

Click it. Your browser opens the live tool.

2 Create the first account

You'll be redirected to the `/login` page.

1. Click **Create one** under the form
2. Fill in name, work email, password (≥ 8 characters)
3. Click **Create account**

You're in. Dashboard shows zero runs (correct — fresh tenant).

3

Lock down public signup once your team is in

By default, anyone with the URL can sign up. Once your QA team has all created accounts, disable public signup:

```
az containerapp update -g ai-platform-auto-tester -n ai-platform-auto-tester --set-env-vars SQA_ALLOW_SIGNUP=false
```

The app rolls out a new revision in ~30 seconds with the flag flipped. From this point on, only existing users can sign in; `/signup` shows a polite "Public signup is disabled" message.

To re-enable later (e.g. when adding a new tester):

```
az containerapp update -g ai-platform-auto-tester -n ai-platform-auto-tester --set-env-vars SQA_ALLOW_SIGNUP=true
```

**Deployment complete.**

You now have a working, multi-user, persistent QA automation tool on Azure with HTTPS. Distribute the URL + the *Tester Guide* PDF to your team. They sign up, run tests, share results.

14 · Day-to-day operations

View live application logs

```
az containerapp logs show -g ai-platform-auto-tester -n ai-platform-auto-tester --follow
```

Streams every Flask request line + every `[run]` / `[login]` / `[q01]` event from the runner. Press **Ctrl+C** to stop.

Push new code from GitHub → Azure

```
cd ~/ai-platform-auto-tester
git pull origin main

# Move local runs/ aside if you've been using local Docker (avoids upload timeout)
mv runs /tmp/runs.bak 2>/dev/null; mkdir -p runs

# Build new image
az acr build --registry aiplatformautotester --image skylar-qa:latest .

# Roll new revision (force pull of new image even though :latest tag is unchanged)
az containerapp update -g ai-platform-auto-tester -n ai-platform-auto-tester --image
aiplatformautotester.azurecr.io/sky-lar-qa:latest --revision-suffix "v$(date +%s | tail -c
5)"

# Restore your local runs (Azure doesn't need them)
rm -rf runs && mv /tmp/runs.bak runs 2>/dev/null
```

Total: ~5 minutes. Zero-downtime — Container Apps keeps the old revision serving traffic until the new one is healthy, then switches.

Restart the app (no code change)

```
REV=$(az containerapp show -g ai-platform-auto-tester -n ai-platform-auto-tester --
query 'properties.latestRevisionName' -o tsv)
az containerapp revision restart -g ai-platform-auto-tester -n ai-platform-auto-tester --
revision "$REV"
```

Scale up CPU/RAM (if QA team grows)

```
az containerapp update -g ai-platform-auto-tester -n ai-platform-auto-tester --cpu 2.0 --
memory 4.0Gi --max-replicas 3
```

Three replicas means three QA runs can execute in parallel. Each replica costs ~₹2,500/mo, so three replicas ≈ ₹7,500/mo.

Connect to Postgres directly

```
PG=$(az postgres flexible-server list -g ai-platform-auto-tester --query '[0].name' -o tsv)
psql "host=${PG}.postgres.database.azure.com port=5432 dbname=auto_tester
user=CelerantAITesting sslmode=require"
# (asks for the password you set in Phase 3)
```

Tear it ALL down

One command kills every resource we created:

```
az group delete --name ai-platform-auto-tester --yes --no-wait
```

⚠ Irreversible.

Deleting the resource group destroys the Postgres database (with all user data and run history), the file shares (with all reports and uploaded files), the Container App, the registry, the image, and the env binding — everything. Make sure to back up anything you care about first.

15 · Cost breakdown

Steady-state monthly cost at QA team usage levels (~50 runs/month, idle most of the day):

RESOURCE	CONFIGURATION	APPROX. ₹/MONTH	APPROX. \$/MONTH
Container App	1 vCPU / 2 GiB · 1 replica · always-on	2,500	\$30
PostgreSQL Flexible Server	Burstable B1ms · 32 GB storage	1,800	\$13
Azure Container Registry	Basic SKU	400	\$5
Storage Account (File Shares)	Standard LRS · ~5 GB used	150	\$1
Log Analytics Workspace	Auto-created · ingestion-priced	~50	\$1
Bandwidth (egress)	~5 GB/mo at QA scale	~50	\$1
Total		~₹4,950	~\$51

Free Trial coverage

New Azure accounts get \$200 in credit valid for 30 days — that covers ~4 months of this stack. After the credit runs out, billing kicks in automatically (you can opt out with one click in the portal).

Cost-cutting options

IF YOU WANT TO SAVE MONEY...	...DO THIS	TRADEOFF
Cut Container App cost in half	Set min-replicas=0 to scale to zero when idle	5–10s cold start on first request after idle period; not ideal for live demos
Skip Postgres backup retention	Reduce backup retention to 7 days (default 7 is already minimum on Flexible Server)	Already minimal — no further savings here
Avoid bandwidth costs	Make sure the QA team uses short report-zip downloads, not streaming	None practical at QA scale; bandwidth is already <\$1/mo
Tear down between sprints	Delete the RG when not actively testing; redeploy when needed	30 minutes to redeploy, must re-create user accounts (data is destroyed)

Setting a budget alert

Catch surprise bills early — set a \$100/month cap that emails you if hit:

```
az consumption budget create --budget-name skylar-qa-budget --amount 100 --time-grain Monthly --start-date "$(date +%Y-%m-01)" --end-date "$(date -v+1y +%Y-%m-01)"
```

16 · Common pitfalls (real ones we hit)

Every one of these came up during the reference deployment. The fixes are baked into the latest code; this section warns you so you recognise them if they recur.

1 Microsoft account exists but no subscription

Symptom: Every `az` command fails with `No subscriptions found for <email>`. Browser portal shows blank "Cost analysis" or no resource list.

Fix: Sign up at azure.microsoft.com/free with a credit card (won't be charged unless trial ends + you upgrade). Subscription activates within 60 seconds. Re-run `az login --use-device-code`.

2 ACR name not globally unique

Symptom: Phase 4 create form shows red error "The registry name is already in use".

Fix: Append digits or your initials to the name — e.g. `aiplatformautotester01`. ACR names share a global namespace across all of Azure.

3 Postgres password rejected

Symptom: Phase 3 create form shows "Password does not meet complexity requirements".

Fix: Azure requires 8–128 chars, with at least 3 of: uppercase, lowercase, digit, symbol. Cannot contain the username (`CelerantAITesting`). Avoid `< > ' " $` to keep CLI usage easy. *Example that works:* `AiTesting2026!`.

4 "Databases" option missing in Postgres sidebar

Symptom: After creating the server, you can't find where to add the `auto_tester` database.

Fix: The option is hidden inside **Settings** in the left sidebar — click to expand, then **Databases** appears. Alternative: use the portal Cloud Shell with `az postgres flexible-server db create -g ai-platform-auto-tester --server-name <your-server> --database-name auto_tester`.

5 az acr build upload timeout

Symptom: `ERROR: ('Connection aborted.', TimeoutError('The write operation timed out'))`

Fix: Move the local `runs/` folder aside before building (it's often 300+ MB of test artefacts that get uploaded even though they're in `.dockerignore`):

```
mv runs /tmp/runs.bak; mkdir runs; az acr build ...; rm -rf runs; mv /tmp/runs.bak runs
```

6 Volume dropdown empty in Container App

Symptom: Phase 9 — clicking **+ Add volume** shows "No file shares found on this Environment".

Fix: You skipped Phase 7 Step 2/3. The file share must be bound at the env level **before** it appears in the app's volume dropdown. Go back to **Container Apps Environment → Settings → Azure Files** and add the binding.

7 Bundle file vanishes after revision rollout

Symptom: User uploaded a Site Search bundle, runs work fine. After redeploying, all bundle requests return 404 — the JS file isn't found.

Fix: You only mounted `/app/runs` persistently, not `/app/data` (where bundles live). Phase 9 of this guide mounts both. If your existing app is missing the data mount, follow Phase 9 Steps 2–3 to add it. Old bundles will need to be re-uploaded once.

8 Site Search timeouts despite valid credentials

Symptom: Every keyword times out with "search_keywords call missing".

Fix: The page's `ssLibrary.init()` is failing to authenticate. Most often: wrong JWT credentials in **Tenant Override**. Test with the demo's `r222ipid` / `W0rkH0us3!` first to verify the flow, then swap in your real tenant credentials.

9 SQL Agent run-sql timeout — endpoint reached but no response captured

Symptom: `generate-sql` returns HTTP 200, then `run-sql` hangs for the full 120s timeout, marked TIMEOUT.

Fix: Often a transient slowdown on the tenant server. If persistent, bump **run-sql timeout** to `300000` – `600000` (5–10 min) under **Advanced timeouts** on the New Run page.

10 Pre-question phase very slow (~3 min before any query runs)

Symptom: After clicking Start QA Run, you wait minutes before the first `[q01]` log line appears.

Fix: The latest code (v3.6+) replaced `wait_until="load"` with `"domcontentloaded"` and removed three `wait_for_load_state("networkidle")` calls (Celerant's SPA polls heartbeats and never goes idle). If on an older version, redeploy with the current image — pre-question time drops from ~227s to ~30-60s.

17 · Troubleshooting / FAQ

Container App won't start — stuck in "Activating"

Check the logs: `az containerapp logs show -g ai-platform-auto-tester -n ai-platform-auto-tester --tail 100`. Most common cause: `DATABASE_URL` typo (especially un-encoded special characters). Re-check Phase 8 Step 4.

"Database not ready yet" page on first load

The app retries the Postgres connection for ~30s on startup. If it still doesn't recover, the Postgres firewall is likely blocking. Re-verify Phase 3 Step 3 — **Allow public access from any Azure service** must be ticked.

Custom domain instead of `*.azurecontainerapps.io`

Add a CNAME in your DNS pointing your domain to the auto-generated FQDN, then:

```
az containerapp hostname add -g ai-platform-auto-tester -n ai-platform-auto-tester --hostname qa.yourcompany.com
az containerapp hostname bind -g ai-platform-auto-tester -n ai-platform-auto-tester --hostname qa.yourcompany.com --environment skylar-qa-env --validation-method CNAME
```

Azure auto-issues a managed certificate (free, auto-renews).

Subscription quota exceeded

Free Trial subscriptions cap regional vCPUs at 4. If you've created other resources today, you may be at the limit. Either delete unused resources or pick a different region (try **westus2** or **eastus**).

The QA tool is up but I can't reach the test target (Celerant tenant)

Tenants on private VPNs aren't reachable from public Azure. Either:

- Coordinate with the client's IT to whitelist the Container App's outbound IP:

```
az containerapp show -g ai-platform-auto-tester -n ai-platform-auto-tester --query 'properties.outboundIpAddresses' -o tsv
```

- Or run the tool on a VM inside the client's VNet

Run is stuck "running" but nothing is happening

If a revision rolled out while a run was in progress, the Playwright thread died but the database row stayed. Have the user click ■ **Stop run** to mark it stopped. Future runs work normally.

How do I view a run's full report locally?

From the run's page, click ↓ **Download report (zip)**. Extract on your laptop, double-click `REPORT.html` — opens in any browser, all screenshots inline, no internet needed.

Reset a user's password

Currently no built-in self-service. Connect to Postgres and update directly:

```
UPDATE users SET password_hash = 'pbkdf2:sha256:600000$...$...' WHERE  
email='user@example.com';
```

Generate the hash with: `python3 -c "from werkzeug.security import generate_password_hash; print(generate_password_hash('newpass1234'))"`

18 · Quick reference card

Resource names (reference deployment)

Resource group	<code>ai-platform-auto-tester</code>
Region	Central US
Postgres server	<code>ai-platform-auto-tester-db-XXXX</code>
Postgres database	<code>auto_tester</code>
Postgres admin	<code>CelerantAITesting</code>
ACR	<code>aiplatformautotester</code>
Storage account	<code>aiplatformautotester</code> (same name OK — different namespace)
File shares	<code>skylar-runs</code> (→ <code>/app/runs</code>) · <code>skylar-data</code> (→ <code>/app/data</code>)
Container Apps env	<code>skylar-qa-env</code>
Container App	<code>ai-platform-auto-tester</code>
Image	<code>aiplatformautotester.azurecr.io/skylar-qa:latest</code>

Most-used CLI commands

WHAT	COMMAND
Sign in	<code>az login --use-device-code</code>
Live logs	<code>az containerapp logs show -g ai-platform-auto-tester -n ai-platform-auto-tester --follow</code>
Build new image	<code>az acr build --registry aiplatformautotester --image skylar-qa:latest .</code>
Roll new revision	<code>az containerapp update -g ai-platform-auto-tester -n ai-platform-auto-tester --image aiplatformautotester.azurecr.io/skyilar-qa:latest --revision-suffix "v\$(date +%s tail -c 5)"</code>
Lock signup	<code>az containerapp update -g ai-platform-auto-tester -n ai-platform-auto-tester --set-env-vars SQA_ALLOW_SIGNUP=false</code>
Tear it all down	<code>az group delete -n ai-platform-auto-tester --yes --no-wait</code>

Test the deployment

1. Open the Container App URL → click **Create one** → sign up
2. **+ New Run** →  SQL AI Engine → fill tenant URL + creds → upload xlsx → Start
3. Or →  Site Search → upload bundle on Test Pages → Start
4. Watch live progress. Open report when done.

Skylar IQ QA Tool · Azure Deployment Guide · v3.6 · May 2026
Internal documentation. Not for external distribution without permission.